Parallel Computing 127 (2026) 103183

Contents lists available at ScienceDirect

Parallel Computing

journal homepage: www.elsevier.com/locate/parco

Microarchitectural comparison, in-core modeling, and memory hierarchy analysis of state-of-the-art CPUs: Grace, Sapphire Rapids, and Genoa☆

Jan Laukemann [a,b,*], Georg Hager [a], Gerhard Wellein [b]

[a] Erlangen National High Performance Computing Center (NHR@FAU), Martensstr. 1, Erlangen, 91058, Germany
[b] Friedrich-Alexander-Universität Erlangen-Nürnberg, Department of Computer Science, Martensstr. 3, Erlangen, 91058, Germany

ARTICLE INFO

Keywords:
Intel sapphire rapids
NVIDIA grace CPU superchip
AMD genoa
Golden cove
Neoverse V2
Zen 4
In-core
Performance analysis
Performance modeling
SpecI2M
Write-allocate evasion
ECM model
Execution-cache-memory model

ABSTRACT

Three big semiconductor companies in HPC are currently competing in the race for the best CPU: AMD, Intel, and NVIDIA. There are significant differences among their state-of-the-art CPU designs, spanning the entire range from instruction execution to cache behavior and main memory bandwidth. In this work, we analyze the performance of CPUs based on the Zen 4, Golden Cove, and Neoverse V2 microarchitectures. We create accurate in-core performance models for use with the Open Source Architecture Code Analyzer (OSACA) tool and compare its prediction accuracy with llvm-mca. Beyond the tool aspect, this reveals interesting differences in in-core design points but also some commonalities. Beyond the single core, we extend our comparison by measuring data-transfer behavior through the memory hierarchy using a variety of microbenchmarks. We thoroughly investigate the "write-allocate (WA) evasion" feature, which can automatically reduce the memory traffic caused by write misses. We show that the Grace Superchip has a next-to-optimal implementation of WA evasion while the Sapphire Rapids CPU can avoid write allocates completely only in specific scenarios. The only way to eliminate WAs on AMD Genoa is the explicit use of non-temporal stores. Finally, we study the cache hierarchy of the CPUs in view of the Execution-Cache-Memory (ECM) performance model, revealing overlapping cache hierarchies on Genoa and Grace in contrast to Sapphire Rapids.

1. Introduction

1.1. Motivation and related work

The Grace Hopper Superchip as well as the Grace CPU Superchip (GCS) mark the first HPC and data center systems by NVIDIA with their own CPU, based on Arm's Neoverse V2 design. One chip comprises 72 cores running at 3.4 GHz all within one ccNUMA domain. With this approach, NVIDIA wants to catch up with the x86 competition and offer a full solution covering both accelerators (i.e., GPGPUs) and hosts (i.e., CPUs). In this work, we analyze the NVIDIA Grace CPU Superchip, compare its performance to the state-of-the-art competitor x86 CPUs Intel Sapphire Rapids (SPR) and AMD Genoa, and provide an in-core performance model for all three microarchitectures for lower-bound runtime prediction that can be used as part of holistic performance models such as Roofline [1], leading to valuable insights into performance bottlenecks of these new CPU architectures.

Although previous work comparing the NVIDIA GCS with competitive x86 and Arm CPUs exists [2,3], it focuses mainly on application performance and ignores microarchitectural details.

While there is a wide range of tools for runtime prediction of machine code via static code analysis, we choose to use the Open Source Architecture Code Analyzer (OSACA) [4,5] as it provides the user with the possibility of adding new microarchitectures into the existing framework relatively easily. Moreover, it stands out as it is not restricted to a single ISA (compared to uiCA [6] and Facile [7]), proved to provide superior accuracy (compared to the LLVM Machine Code Analyzer llvm-mca [8]) for already existing performance models, focuses on performance prediction rather than the code quality (compared to the Code Quality Analyzer (CQA) [9]), and provides a white-box model for more insight for the user (compared to AI-based tools like Ithemal [10] and GRANITE [11]). In previous work [12] a thorough analysis on write-allocate evasion for Intel Ice Lake and Sapphire Rapids CPUs was conducted. In this work we broaden the view to AMD Genoa and the NVIDIA Grace CPU Superchip.

The work presented here is an extended version of a short paper published at the PMBS 2024 workshop [13]. That paper investigated the architectural differences and peculiarities of the three state-of-the-art server models Intel Sapphire Rapids, AMD Genoa, and NVIDIA

☆ This article is part of a Special issue entitled: 'PMBS Workshop' published in Parallel Computing.

* Corresponding author at: Friedrich-Alexander-Universität Erlangen-Nürnberg, Department of Computer Science, Martensstr. 3, Erlangen, 91058, Germany.
E-mail address: jan.laukemann@fau.de (J. Laukemann).

https://doi.org/10.1016/j.parco.2026.103183
Received 11 July 2025; Received in revised form 17 October 2025; Accepted 13 January 2026
Available online 29 January 2026
0167-8191/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Grace CPU Superchip and focuses on the construction of an accurate in-core performance model for use with the Open Source Architecture Code Analyzer tool and the comparison of its prediction accuracy with llvm-mca. We extended our original work by a thorough study of the write-allocate evasion feature on Sapphire Rapids, further insights on the in-core execution of the Neoverse V2 and Golden Cove microarchitectures, and an extensive analysis on the memory hierarchy of the NVIDIA Grace CPU Superchip and the construction of a naive Execution-Cache-Memory (ECM) performance model for it.

This paper is structured as follows: Sections 1.2 and 1.3 give an overview of CPU port models and the testbed of this work. In Section 2, we analyze the microarchitectures of the three CPU models, starting from the single-core performance capabilities and basic clock frequency throttling for SIMD-heavy code, up to building a performance model of the in-core executing for OSACA and validating it against the state-of-the-art static code analyzer llvm-mca. Next we evaluate the write-allocate evasion capabilities of the three servers on both the core as well as chip level with a special focus on the Sapphire Rapids node. In Section 3, we extend the investigation to the memory hierarchy by constructing the components of an ECM performance model, conducting several microbenchmarks to fix the model parameters. Finally, in Section 4, we summarize our findings and define the next potential steps for future work.

1.2. Brief overview of the in-core port models

When thinking about the performance of a single CPU core, we assume what is widely known as the *port model*: Each instruction, optionally split into one or more *micro-ops* (μ-ops), gets assigned to and processed by *execution units* (EUs) and may even require multiple EUs (e.g., to load data and do an arithmetic computation on the loaded value). On the other hand, one EU might exist multiple times and can thus increase the instruction throughput via out-of-order (OoO) execution; e.g., two FMA units that can be accessed in parallel double the throughput for this instruction type. One or more EUs are grouped behind a *port* as seen by the scheduler, i.e., for each port and each cycle, one μ-op can be issued (with a global maximum number of μ-ops issued per cycle). Fig. 1 shows the port model of the Neoverse V2 microarchitecture, used in NVIDIA's Grace CPU. While the width of the SVE registers is relatively small (128 bit), there is considerable instruction level parallelism (ILP) in available ports with similar execution units. As often seen in modern OoO-architectures, the execution of instructions including floating-point data is separated from the execution of integer data. More information on the idea of port models can be found in previous work [4].

1.3. Testbed and experimental methodology

All experiments were carried out on dedicated servers in our test cluster: A two-socket NVIDIA Grace CPU Superchip, a two-socket Intel Xeon Platinum 8470, and a two-socket AMD EPYC 9684X system. The specific hardware features are listed in Table 1.

For compilation we used GCC 12.1, the oneAPI 2023.2 compiler framework and LLVM Clang 17.0.6 for the x86 machines and the Arm C Compiler 23.10 (based on LLVM 17) and GCC 13.2 for the Grace server. For cycle-accurate measurements we set the clock frequency to the corresponding base frequency using SLURM [15] if possible, i.e., 2.0 GHz for SPR and 2.55 GHz for Genoa. While Grace does not allow frequency fixing, we could not observe any frequency change running our benchmarks and validated the clock frequency of all runs with hardware performance counters using LIKWID [16] 5.3.0 [17].[1]

[1] As there is no official release of the LIKWID software with NVIDIA Grace support and only limited support for Genoa, we used the development versions from PR585 and PR618, respectively.

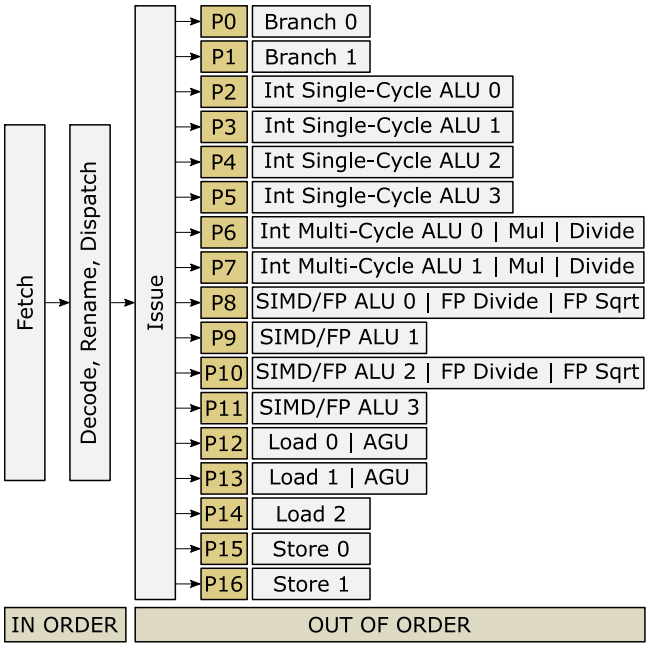


Fig. 1. Arm Neoverse V2 core block diagram and port model, compiled from Arm's Software Optimization Guide [14]. The port numbering ("P[0-9]+") is chosen by the authors and is not part of the official documentation.

This tool was also used for all other hardware performance counter measurements. To validate our in-core performance models, we used the OSACA [4,5] tool in version 0.5.3 including our own extensions (which will be part of the next release) for supporting the microarchitectures in this paper. Furthermore, the LLVM Machine Code Analyzer (llvm-mca) [8] was used for comparing the accuracy of our model.

2. Architectural analysis

2.1. Hardware model and instruction performance

Since a port model visualization of all three microarchitectures as shown in Fig. 1 would go beyond the scope of this work and can also be found in the respective optimization manuals [18,19], we show only the key aspects of the three cores in Table 2.

While Golden Cove and Zen 4 have approximately the same number of ports (12 and 13, respectively), the Neoverse V2 stands out with its 17 ports, fully offloading any non-floating-point operations to other ports and providing a high ILP. As a downside, even though the core supports the SVE vector extension for width-agnostic vector registers, the maximum register width is 128 bit, which is only a fourth of Golden Cove's 512 bit registers. This leads to the expectation that the Golden Cove architecture can show its strength when executing highly vectorized code while the Neoverse V2 shines with code that is hard to vectorize and has many scalar instructions, as often seen in data center and AI workloads.

We further noticed that the Neoverse V2 achieves its maximum L1-register throughput of three loads per cycle or two stores per cycle only if NEON data move instructions on quad-word registers (e.g., `ldr q, mem`) are used, while the L1 bandwidth drops to 32 byte/cy for loading and 16 byte/cy for storing when using SVE data transfer instructions (e.g., `ld1d z, p, mem`). Note also that the core has three address generation units (AGUs), only two of which can be used for store addresses.

The Zen 4 meets the two extremes in the middle with 256-bit registers and slightly more ILP than Golden Cove. Even though Zen 4 supports the AVX-512 extensions, their execution is split into 2×256 bit

Table 1
Comparison of the core features of the Grace CPU Superchip (GCS), the Intel Xeon Platinum 8470 (SPR), and the AMD EPYC 9684X (Genoa). For all servers, the L1 and L2 cache are exclusive caches per core, while the L3 is shared within one chip.

	LLLL NVIDIA Grace Superchip "GCS"	Intel Xeon Platinum 8470 "SPR"	AMD EPYC 9684X "Genoa"
Cores	72	52	96
Base Frequency	3.4 GHz	2.0 GHz	2.55 GHz
Max. Frequency	3.4 GHz	3.8 GHz	3.7 GHz
Theor. DP Peak	3.92 Tflop/s	6.32 Tflop/s	8.52 TFlop/s
Achiev. DP Peak	3.82 Tflop/s	3.49 Tflop/s	5.1 TFlop/s
TDP	250 W	350 W	400 W
L1 cache size	64 KB	48 KB	32 KB
L2 cache size	1 MB	2 MB	1 MB
L3 cache size	114 MB	105 MB	1152 MB
Main memory	240 GB	512 GB	384 GB
	LPDDR5X	DDR5	DDR5
ccNUMA domains	1	4 (SNC-mode)	4 (NPS=4)
Theor. max. mem BW	546 GB/s	307 GB/s	461 GB/s
Meas. max. mem BW	467 GB/s	273 GB/s	375 GB/s

Table 2
Comparison of the in-core features and port models for the GCS, SPR, and Genoa cores.

	LLLL GCS (Neoverse V2)	SPR (Golden Cove)	Genoa (Zen 4)
Number of ports	17	12	13
SIMD-width	16 B	64 B	32 B
Int units	6[a]	5	4
FP vector units	4	3	4
Loads/cy	3 × 16 B	2 × 64 B	2 × 32 B
Stores/cy	2 × 16 B	2 × 64 B	1 × 32 B

[a] GCS's 6 Int ports comprise 2 multi-cycle + 4 single-cycle ports.

packets. While a comparison of the sustained peak memory bandwidths heavily depends on the built-in memory type and the number of DIMMs and would not represent a fair competition, we can compare them with the theoretical maximum and state that Genoa only achieves 81% of its theoretical memory bandwidth peak, while GCS and SPR reach 87% and 90%, respectively.

Although there is some documentation on the microarchitectures' backends [14,18,20], the information is often incomplete or insufficient to build a useful performance model. Therefore, we write microbenchmarks with various benchmark tools [21,22] for every interesting instruction to obtain its throughput, latency, and port occupation. For the latter, it is often necessary to interleave the instruction with known instructions to infer the potential ports of execution. While each model comprises hundreds of entries of individual combinations of assembly instructions and operands, we show the throughput and latency for some of the most important double-precision instructions in Table 3. While the "VEC" rows refer to the common SIMD instructions in x86 and AArch64, such as `v[add, mul, fmadd213, div]pd` and `f[add, mul, mla, div]`, respectively, the "scalar" rows refer to scalar assembly instructions from recent ISA extensions working on vector registers, such as `v[add, mul, fmadd213, div]sd`, `f[add, mul, div]`, and `fmadd`. However, these numbers often fit the throughput and latency of similar or equivalent instructions from the respective ISA, e.g., when including a negate, a subtract instead of add, additional masking, or instruction aliases.

The Golden Cove architecture shows the highest throughput for all shown vector instructions due to its large register width. The Neoverse V2 can demonstrate its strength for scalar instructions due to the large ILP. A single Zen 4 core is slower or breaks even in terms of throughput for all instructions shown; however, a full chip comprises 96 cores while an SPR chip comes only with 52 cores. Therefore, if an application allows a high parallelism on the node level, e.g., through OpenMP, the overall throughput of the Genoa system might come out first, as shown in the artificial peak FLOP benchmark used in Table 1. When looking at the latencies of the investigated instructions, one can clearly observe the superiority of the Neoverse V2 which shows

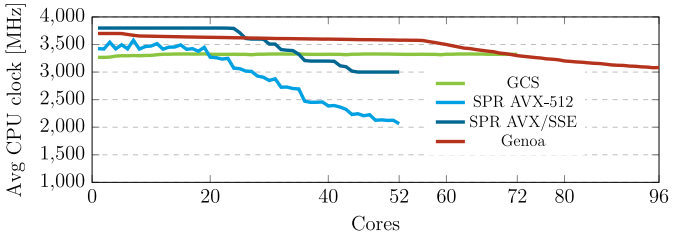


Fig. 2. Sustained CPU clock frequency for arithmetic-heavy code on GCS, SPR, and Genoa across one chip. If no ISA extension is specified, the architecture could sustain the same frequency for all supported ISA extensions.

a lower or even latency for every single instruction in Table 2. Thus, arithmetic-heavy latency-bound codes such as iterative solvers using the Gauss–Seidel method [23] could benefit from running on a Grace CPU Superchip compared to the two competitors. Especially Intel seems to trade off their high throughput performance against a relatively high instruction latency, even though they managed to decrease the ADD latency by half compared to the predecessor Ice Lake microarchitecture.

2.2. Clock frequency throttling

Although wide registers can provide a good out-of-the-box speedup when using vectorization on a single core, it is a well-known problem for Intel to require the cores to throttle down for AVX-512-heavy code and when using multiple cores due to thermal constraints. Therefore we analyzed the sustained clock frequency for arithmetic-heavy codes while scaling across a socket on all systems (see Fig. 2). Each benchmark ran for several minutes and the clock frequency of all active cores was tracked using hardware performance counters. While SPR shows a different behavior right from the start for AVX-512-heavy code, the sustained frequency for the GCS and Genoa did not change across ISA extensions. Both SPR and Genoa eventually fall down to

Table 3
Throughput and latency for some double-precision instructions on GCS, SPR, and Genoa. If multiple values for one instruction were applicable, e.g., due to different performance for different vector widths, the best performance (i.e., highest throughput, lowest latency) was selected. Note that the throughput of the "gather" as a load instruction is given in "cache lines per cycle" while the rest is evaluated in double precision elements per cycle.

	LLLLLL GCS (Neoverse V2)	SPR (Golden Cove)	Genoa (Zen 4)	GCS (Neoverse V2)	SPR (Golden Cove)	Genoa (Zen 4)
Instruction	Throughput [elements/cy]			Latency [cy]		
gather [CL/cy]	1/4	1/3	1/8	9	20	13
VEC ADD	8	16	8	2	2	3
VEC MUL	8	16	8	3	4	3
VEC FMA	8	16	8	4	4	4
VEC FP Div	0.67	0.5	0.8	7	14	13
Scalar ADD	4	2	2	2	2	3
Scalar MUL	4	2	2	3	4	3
Scalar FMA	4	2	2	4	5	4
Scalar Div	0.4	0.25	0.2	12	14	13

a frequency of 2.0 GHz and 3.1 GHz for AVX-512-heavy code, which results in 53% and 84% of their respective single-core turbo limit, even though SPR manages to sustain a frequency of 3.0 GHz for the case of AVX- or SSE-heavy code (78% of Turbo). The NVIDIA GCS exhibits a constant frequency of 3.4 GHz (the base frequency) throughout the whole socket. Therefore, for highly parallel arithmetic-heavy code, one might see better performance on GCS compared to SPR despite even for throughput- or latency-bound code due to a 1.7× higher sustained clock frequency.

2.3. Performance modeling with OSACA

The individual measurements as shown in Table 3 can be incorporated with the specific port occupations into an in-core performance model that can be used for optimistic runtime prediction or as a building block for node-wide performance models (e.g., a more realistic horizontal ceiling in the Roofline Model [1] or the in-core component of the Execution-Cache-Memory (ECM) Model [24]). The Open Source Architecture Code Analyzer (OSACA) provides such an in-core model based on static analysis of assembly code without compiling or running it. For accurate predictions, OSACA applies several assumptions: The kernel is an inner-most loop and it is sufficient to consider the steady-state, i.e., ignoring any wind-up or cool-down phases, which does not decrease prediction quality for the vast majority of scientific applications as they are dominated by a few hot spot loops. Conditional paths can be modeled by isolating different control flows and building an upper and lower limit based on the available branches. Finally, as OSACA models the upper roof for in-core performance, it assumes all data to be in the L1 cache and perfect out-of-order scheduling. If these requirements are met, OSACA can predict the runtime of any assembly kernel with satisfying accuracy. The results of this work can thus be easily applied to more complex cases. In Section 3 we show how the model can be used as a component of a more complete performance model considering the full memory hierarchy. The tool delivers three different predictions: (a) an optimistic throughput analysis considering the port pressure on each individual port (i.e., the reciprocal throughput of all instructions executed on a port considering perfect scheduling), (b) a loop-carried-dependency analysis, trying to detect dependency chains across loop iterations and showing the overall runtime (i.e., latency) of each one, and (c) a critical path analysis, detecting the longest dependency chain within one loop, which can cause slowdowns due to no full overlap of dependency chains for a very long critical path or a small number of loop iterations. As a runtime prediction, we use the maximum number of cycles (i.e., slowest runtime) out of the throughput analysis and the loop-carried-dependency analysis as suggested in previous work on this tool [25].

For validation of our models we used 13 streaming microbenchmarks (Jacobi [2D 5-point|3D 27-point|3D 7-point|3D 11-point] stencil, ADD, COPY, Gauss–Seidel 2D 5-point stencil, π-computation by integration, INIT, Schönauer Triad, Sum reduction, STREAM Triad [26], UPDATE), compiled with different compilers (Armclang, GCC, oneAPI, and Clang) and different optimization flags (`-O1`, `-O2`, `-O3`, and `-Ofast`), resulting in 416 tests and 290 unique assembly representations.

Fig. 3 (based on graphs in related work [27]) shows histograms of the relative prediction error (RPE) for the kernels with our models of the investigated microarchitectures incorporated into OSACA versus the LLVM performance models in llvm-mca. We compare our model against llvm-mca as it is a widely-known state-of-the-art static code analyzer and—to the best of our knowledge—the only available tool in this field besides OSACA that also supports non-Intel and Arm-based microarchitectures. Each bucket marks a range of 10% relative error; bars right of the red dotted zero line indicate a prediction faster than the actual measurement while bars left of the line indicate a slower prediction. The bucket in the very left collects all predictions larger than −1.0 (i.e., off by more than a factor of 2). As we aim to provide a lower-bound estimate, we prefer to see all errors on the right of the zero line. Except for a few versions of the Gauss–Seidel kernel for the Neoverse V2, where OSACA (correctly) predicts a register dependency that the CPU can overcome by register renaming, and the π kernel for Zen 4, where our model assumes a lower throughput for the scalar divide than we measure, this is the case for all other tests (96%) with our performance model. There is one kernel predicted incorrectly by more than a factor of 2, and 37% are predicted accurately with a positive RPE of less than 10%. Similarly, 44% of the kernels are predicted with an accuracy of less than 20% positive RPE. The llvm-mca model, however, predicts 75% of the test kernels slower than the actual measurements, with 14 measurements being off by more than a factor of 2. Only 10% (16%) are predicted correctly with a positive RPE of less than 10%, although this value increases to 32% when considering the 10% bucket on the negative side of the zero line. When considering all predictions with less than 20% RPE, the number of kernels slightly grows to 16% considering a positive RPE and 48% when including kernels with negative RPE larger than −20%.

The average RPE of only the under-predictions (i.e., right-hand-side errors) of our model in OSACA shows a smaller error for Golden Cove, V2, and Zen 4 with 24%, 30%, and 18% versus the LLVM model showing 38%, 34%, and 20%. When looking at the global (i.e., absolute) RPE, our model still performs better for Golden Cove (30% vs 35%) and V2 (26% vs 52%), and is slightly worse for Zen 4 than the LLVM-based model (18% vs 16%).

2.4. Write-allocate evasion

Looking beyond the CPU core towards data movement in the memory hierarchy, one interesting feature that has entered x86 processors with the Intel Ice Lake generation is the automatic evasion of write-allocate (WA) transfers from memory. Write-allocate usually occurs in cache-based architectures when a standard store operation from

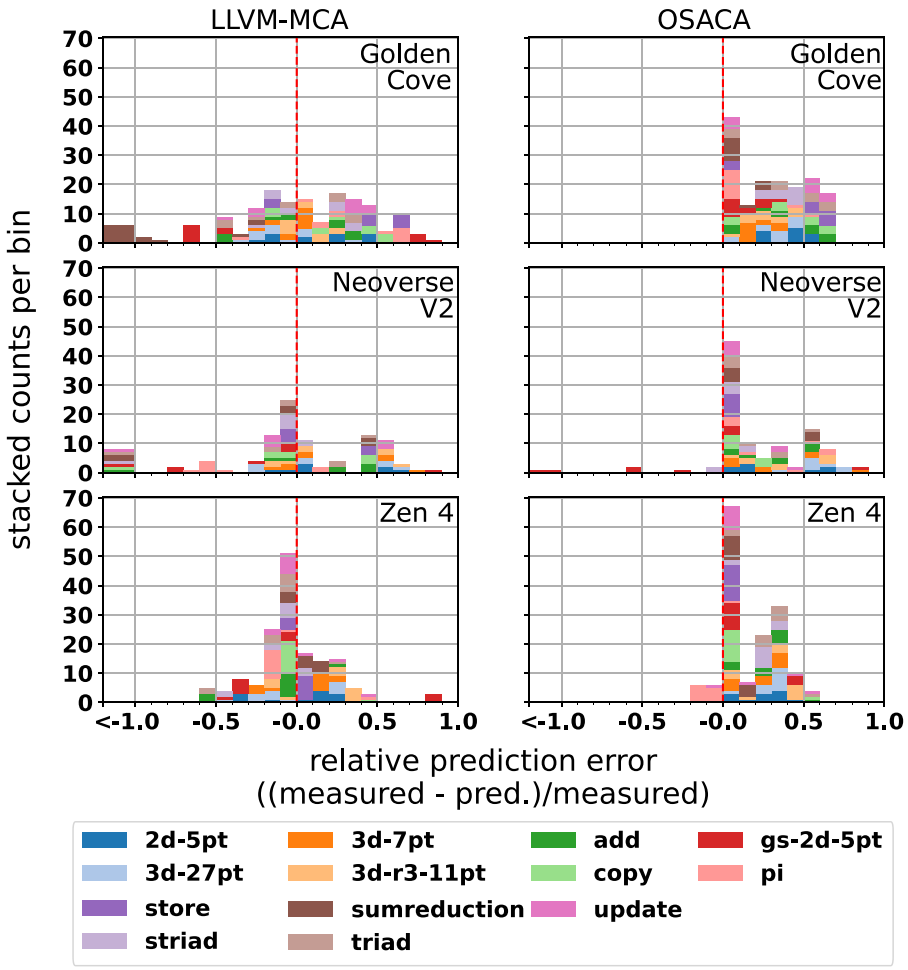


Fig. 3. Relative prediction error of 416 test blocks for llvm-mca and OSACA. Bars right of the red dotted line indicate a prediction faster than the actual measurement while bars left of the line indicate a slower prediction.

a register to memory causes a write miss: Since the core can only communicate with its L1 cache, the cache line must be read from memory before it can be modified and then (later) written back. This extra data traffic can impact the performance and clutter the cache with data that may not be needed soon. *Cache line claim* and *non-temporal stores* are two ways to avoid write-allocates. Both can be supported by special instructions that claim a cache line in the cache without reading it first (available on some Arm CPUs) or write data to memory through a special write-combine buffer outside the normal cache hierarchy (available on Arm and x86 CPUs). Cache line claim can also be automatic if a core is able to detect that a cache line will be overwritten entirely. This feature has been supported for a long time by many Arm CPUs (including, e.g., the Marvell ThunderX2 and GCS) and by Intel server chips starting with the Ice Lake family, where Intel termed it *SpecI2M* [28].

In order to fathom the ability of the CPUs under investigation to employ automatic and explicit WA evasion, we run a simple store-only (array initialization) benchmark, measure the actual memory data traffic (which includes write-allocates), and divide it by the amount of stored data. With perfect WA evasion in place, this ratio should be equal to one. It should be equal to two if the full WA transfers apply. On Intel CPUs it was shown previously [12,29] that the efficiency of SpecI2M depends crucially on the saturation of the memory interface: Only if a significant fraction of the maximum memory bandwidth is utilized will the WA evasion mechanism kick in. Fig. 4 shows the results of the store benchmark with respect to the number of cores utilized for all three CPUs. In case of SPR and Genoa, we have added a variant

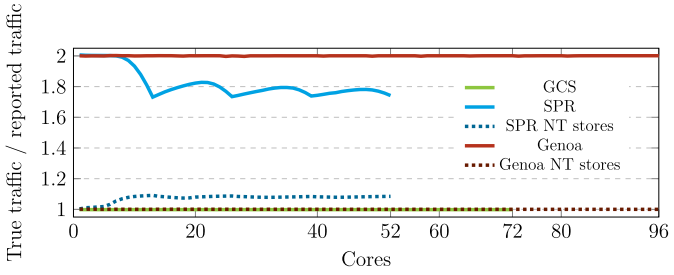


Fig. 4. Ratio of actual memory traffic to stored data volume vs. number of cores for a store-only benchmark loop (working set 40 GB). A value of 1.0 indicates perfect WA evasion, while a value uf 2.0 indicates full WA traffic. The variants labeled "NT stores" use non-temporal store instructions, while the others use standard stores. "Compact" thread-core affinity was used in all cases.

with non-temporal (NT) stores for reference; ideally, NT stores should eliminate the WA transfers entirely. They are, however, not automatic; the compiler (or the programmer) must employ them specifically.

The results show that only GCS is able to completely avoid WA transfers automatically in this simple scenario (solid green line). The SpecI2M mechanism in SPR can only reduce write-allocates by up to 25% and only kicks in when a large part of the 13 cores on a ccNUMA domain is utilized (solid blue line). The only way to avoid WAs on Genoa is via non-temporal stores, which works perfectly (dotted dark

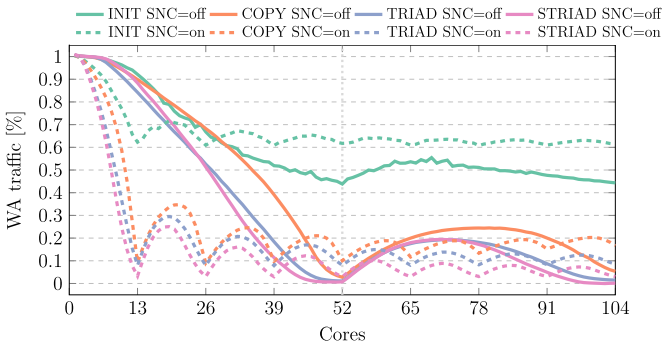


Fig. 5. Write-Allocate study on a Intel Xeon "Sapphire Rapids" node for different streaming benchmark loops (working set 7.7 GB per array). The *y*-axis shows the WA traffic, with 0 meaning all WAs have been avoided and 1 meaning all stores led to a corresponding WA. We show the minimum out of five runs. Compact thread-core affinity was used, populating consecutive cores as the number of threads is increased.

red line vs. solid red line). Finally, on SPR even the non-temporal stores are not 100% effective, and there is a residual 10% of WA traffic except for very small core counts (dotted blue line).

As the SPR node is the only architecture showing an unexpected behavior with its rather inefficient cache line claim, we conducted further experiments on the write-allocate evasion for different streaming kernels subject to this optimization: the Schönauer Vector Triad ($a[\,] = b[\,] + d[\,] \times c[\,]$), the STREAM Triad ($a[\,] = b[\,] + s \times c[\,]$), a COPY ($a[\,] = b[\,]$), and an INIT ($a[\,] = s$, as shown before in Fig. 4, for completeness) benchmark. For each kernel and per iteration, there needs to be 8 bytes (one double precision element) stored and 24, 16, 8, or 0 bytes loaded for the Schönauer Triad (STRIAD), STREAM Triad (TRIAD), COPY, and INIT, respectively. If a write-allocate occurs, there will be an additional 8 bytes of read data per iteration. We measure the corresponding data volume to and from main memory via performance counters and show the write-allocate traffic versus the number of cores for Sapphire Rapids in Fig. 5. For a better understanding of the SpecI2M feature, we run the experiment on the node in two different settings: (i) Sub-NUMA Clustering (SNC) turned off, i.e., with one ccNUMA domain per socket, and (ii) SNC=4, i.e., with four ccNUMA domains per socket. While the INIT (i.e., store-only) kernel shows the known inefficient WA evasion with, in the best case, 61% of all writes triggering a read operation, we can observe a much better WA evasion for kernels with additional read streams. The three other benchmark loops are much better at avoiding WAs, with a trend to higher efficiency with more load streams: While COPY with only one load stream manages to avoid 91% of the WA traffic in the best case for SNC=on, TRIAD can avoid 92% and STRIAD even 97% with two and three individual load streams, respectively.

Furthermore, the dependency of the efficiency of SpecI2M on memory bandwidth saturation becomes clear when comparing SNC=on (four ccNUMA domains per socket) with SNC=off (one ccNUMA domain per socket). Even though the best-case scenario for avoiding WAs improves slightly to 97%, 99%, and 99% for COPY, TRIAD, and STRIAD, respectively, the effect kicks in a much slower manner as more cores are needed to get anywhere near bandwidth saturation. We thus speculate that the SpecI2M feature is coupled to *Dynamic Prefetch throttling* [28], which reduces the prefetch distance at high bandwidth.

It seems that especially store-dominated loops (like INIT) benefit from SNC=off as the WA traffic with a drop to 44% is more prominent than in all other cases. In all experiments, we see the increase of WA traffic with the beginning of a new ccNUMA domain and, therefore, less pressure on the memory controllers. This effect appears again to be slightly more dominant when there is already a high ratio of avoided WAs and there are fewer load streams, as the COPY kernel shows a significant rise in WAs when accessing a new ccNUMA domain.

3. Cache hierarchies

3.1. Execution-Cache-Memory model

The Execution-Cache-Memory (ECM) model [23,24] is an analytical performance model for streaming loop kernels with regular data-access patterns, using first principles and machine-dependent constraints. Unlike the Roofline model (RLM), the ECM model can identify execution and data transfer bottlenecks for single-threaded programs and predict the scaling behavior of kernels across the cores of a multi-core chip. While the RLM always assumes full overlap between data transfer and instruction execution, the ECM model allows to differentiate between overlapping or non-overlapping time contributions within the cache hierarchy.

The ECM model contains a number of time contributions for the CPU architectures under consideration: T_{OL} is the execution time for instructions that can overlap with data transfers through the memory hierarchy. T_{nOL} is the time (cycles needed for retirement) for instructions that does not overlap with data transfers. Which instructions count against which of these contributions depends on the architecture. Beyond execution, the ECM model considers data transfer times T_{ij} for data transfers between memory hierarchy levels i and j, e.g., T_{L1L2} is the transfer time for data between L1 and L2 caches. If necessary, these could be split into read and write parts, i.e., if the load bandwidth is different from the write bandwidth. In order to calculate these data transfer times, only the bandwidth of the data path is taken into account, i.e., $T_{L1L2} = V_{L1L2}/b_{L1L2}$, where V_{L1L2} is the data transfer volume and b_{L1L2} is the bandwidth. Usually we calculate the data volumes (and times) for a fixed number of iterations in a loop; eight is a good number for loops handling double-precision data since all numbers come out as conveniently small integers. For example, if three cache lines are transferred between L1 and L2 for executing eight iterations, $b_{L1L2} = 64$ byte/cy, and the L1-L2 data path is half-duplex, we have $T_{L1L2} = 192\,\text{byte}/(64\,\text{byte/cy}) = 3$ cy. If necessary, latency cycles can be added to the data transfer times if the pure bandwidth does not yield a satisfactory model.

In order to predict the serial runtime T_{ECM} of the loop, all time contributions must be combined via an *overlap model*: $T_{ECM} = f\left(T_{OL}, T_{nOL}, \{T_{ij}\}\right)$. For Intel server CPUs up to the Broadwell generation, a largely non-overlapping assumption works best, i.e.,

$$f\left(T_{OL}, T_{nOL}, \{T_{ij}\}\right) = \max\left(T_{OL}, T_{nOL} + T_{L1L2} + \cdots\right) ,$$

but other architectures have more overlapping characteristics [23,30]. Finding the appropriate form of $f(\cdot)$ is the main challenge in constructing the ECM model. Note that not all relevant features in the cache hierarchy are documented by vendors; if necessary, bandwidth numbers must be inferred from measurements.

Optimistically, the prediction for multiple cores assumes perfect scalability until a bandwidth bottleneck (such as memory bandwidth or the L3 bandwidth) is hit. This model can be refined to yield exceptional accuracy for some architectures [31]. In this paper, we restrict ourselves to modeling the sequential execution time.

While the full construction of three ECM models for the different microarchitectures tackled in this work would go beyond the scope of this paper, we present the preliminary investigations needed for basic cache assumptions based on the NVIDIA Grace CPU Superchip and show the fundamental measurements necessary for constructing an ECM model for all three CPU models.

3.2. Inclusive or exclusive L3 cache

As opposed to the inclusive L2 cache, which is part of the official Neoverse V2 core IP and well documented [32], the design of the L3 cache is up to NVIDIA and is not disclosed in any publicly available documentation. Fig. 6 shows reciprocal performance in cycles per iteration for a parallel DAXPY benchmark on one, four, 16, and 64

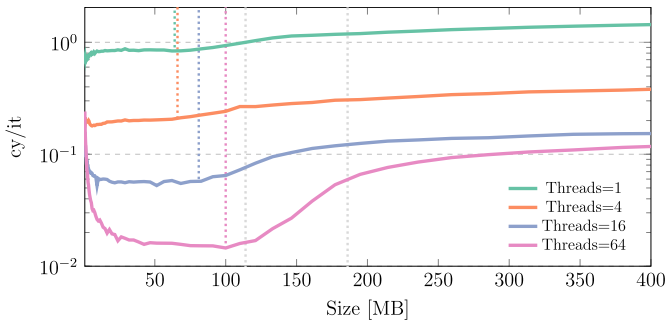


Fig. 6. The parallel DAXPY kernel (inverse performance in cycles per iteration) on the NVIDIA Grace CPU Superchip for different thread counts with working set sizes in the order of the L3 cache size. The left vertical dotted line in light gray represents the size of the L3 cache (114 MB) while the right vertical dotted line in light-gray represents the size of the L3 cache in addition to the L2 caches (114 MB + 72 × 1 MB = 186 MB).

threads for a data set size in the order of the L3 cache size, which is indicated by the left light-gray vertical dotted line at 114 MB. If the L3 cache was an *inclusive* cache, all four data series should drop out of L3 at the same data set size, indicated through a lower performance and, hence, a higher cy/it value. We can see that all benchmarks slowly start to drop out of L3 already before reaching the point of 114 MB, most likely due to non-optimal replacement strategies, but the knee point gets shifted to the right with an increasing number of threads. This indicates an *exclusive* L3 cache, since more threads mean also more L2 cache, which must be added on top of the original L3 cache size before dropping out to main memory. This results in a maximum data set size before falling back into main memory of 186 MB, i.e., the sum of all L2 and L3 cache sizes (indicated by the right light-gray vertical dotted line). The thus predicted knee point for each run is marked with a vertical line in the corresponding color of the data series.

3.3. Memory hierarchy benchmarking

As Intel and AMD provide information on the exclusivity of their last-level caches in their documentation [18,19], we can continue to apply the overlap hypotheses for the CPUs investigated. Unfortunately, all three hardware vendors disclose little to no information about how their memory hierarchies are organized. Some of the relevant data (e.g., inter-cache data path bandwidths) must thus be inferred from benchmarks. We use measurements of the benchmark kernels INIT (`a[i] = s`), COPY (`a[i] = b[i]`), and SDAXPY (Schönauer DAXPY, `a[i] = a[i] + c[i] * b[i]`), as shown in Fig. 7.

Each subplot shows the minimum, median, and maximum time of the benchmark in cycles per iteration out of 30 runs across the memory hierarchy from L1 to main memory, grouped column-wise by CPU model. The cache capacity limits are marked with vertical lines along the x-axis. The red line represents the naive ECM model retrieved from the assumptions taken in the following part of the section.

The predicted runtime for data in the L1 is given by OSACA as acquired in Sections 2.1 and 2.3 and matches the measurements in all cases close to perfectly. For NVIDIA GCS, we assume a full overlap (i.e., a "max" function) between L1 and the other cache levels and quantify the data transfer bandwidth from and to L2 as 48 byte/cy half-duplex (i.e., either read or write). We measure an L3 bandwidth of 32 byte/cy half-duplex and assume its data transfer cannot overlap with the store data transfer from L1 to L2; i.e., when data is written to the L2, we cannot read from nor write to the L3. Finally, we assume a bandwidth of 137 byte/cy half-duplex for data transfers from and to the main memory, non-overlapping with the cache data transfer. The overall ECM model can be therefore formulated as:

$$T_{\text{ECM,GCS}} = \max\left(T_c, T_{L1,LD}, T_{L1,ST}, T_{\text{L1L2}}, T_{\text{L1L2,ST}} + T_{\text{L2L3}} + T_{Mem}\right)$$

with T_{C}, $T_{\text{L1,LD}}$, and $T_{\text{L1,ST}}$ being the in-core predictions retrieved by OSACA and T_{L1L2} and T_{L2L3} being the transfer times for all loads from and stores to L2 and L3, respectively. $T_{\text{L1L2,ST}}$ refers to the time for all data evicted from L1 to L2.

We notice that especially in the transition regions between L2/L3 and L3/main memory, the measurements get more volatile and lead to larger discrepancies between the minimum and maximum. In particular the in-memory predictions are too optimistic, indicating that a latency penalty applies. Furthermore, it is unknown how the segmented L3 cache behaves w.r.t. data traffic to a local vs. a remote core. And finally, although write-allocate evasion works for data in memory, it is unclear whether it also applies to in-cache data and at which cache level a cache line is claimed. In comparison to the same measurements on AMD or Intel platforms we observe more gentle increases in runtime when dropping out of any cache. This is partly due to the nature of an exclusive cache (and partially visible for the exclusive L3 caches of SPR and Genoa as well), but there are clearly optimization techniques and cache-eviction strategies employed by the hardware to keep useful data as long as possible in the nearest cache.

For the measurements on the Intel SPR in the middle column, we can immediately see more prominent distinctions between the cache levels and barely any variation except for some outliers at the threshold between L3 and main memory. For the SPR, similar to previous ECM model hypotheses for Intel processors, we assume no overlap between any memory hierarchy and a half-duplex data path with 64 byte/cy and 32 byte/cy bandwidth between L1 and L2 and between L2 and L3, respectively. As main memory bandwidth we use the 68 Gbyte/s per ccNUMA domain as measured in Section 1.2, resulting in 34 byte/cy at 2 GHz. This overlap hypothesis seems to fit well overall as a general prediction, equally over- and underestimating parts of the measurements and precisely predicting most of the cache levels. The formulated ECM hypothesis can be written as follows:

$$T_{\text{ECM,SPR}} = \max\left(T_c, \max(T_{L1,LD}, T_{L1,ST}) + T_{\text{L1L2}} + T_{\text{L2L3}} + T_{Mem}\right)$$

The previously-known capability of AMD of overlapping data transfers in the memory hierarchy [31] becomes clearly noticeable by comparing the range of execution times between in-core and in-memory data sets against the GCS and SPR systems. For Genoa's ECM hypothesis, we assume a full-duplex 32 byte/cy data path between L1 and L2, as well as a full-duplex 24 byte/cy connection between L2 and L3, and full overlap of all data transfers between caches and registers. Data transfers to and from main memory cannot overlap and occur at 94 Gbyte/s, corresponding to 47 byte/cy at 2 GHz. We speculate that the observed plateaus in the L3 cache are caused by AMD's 3D V-Cache technology with stacked L3 cache tiles, which allows faster L3 access in the first 32 MB of the L3 while the spread-out L3 cache, local to each of the three CCXs (Core Complexes) per ccNUMA domain, gives rise to a very slow decrease of bandwidth when exceeding the L3 size of 96 byte per CCX. One can see that the full-overlap assumption in the caches fits excellently to the measured performance data, although we can see some deviation for data in the memory. Thus we assume there exists some enhanced prefetching into the L2 for explicit reads (i.e., reads not triggered by write-allocates) from main memory, which needs further investigation. Therefore, the overlap hypothesis can be summarized as follows:

$$T_{\text{ECM,Genoa}} = \max\left(T_c, T_{L1,LD}, T_{L1,ST}, T_{\text{L1L2}}, T_{\text{L2L3}}\right) + T_{Mem}$$

We are confident that, with more insight on the hardware, e.g., through more measurement data or vendor information on replacement strategies, these ECM models can be further improved.

4. Conclusion

Via a thorough in-core analysis of the NVIDIA Grace CPU Superchip, the Intel Sapphire Rapids, and the AMD Genoa CPU, we showed peculiarities of their microarchitectures Neoverse V2, Golden Cove,

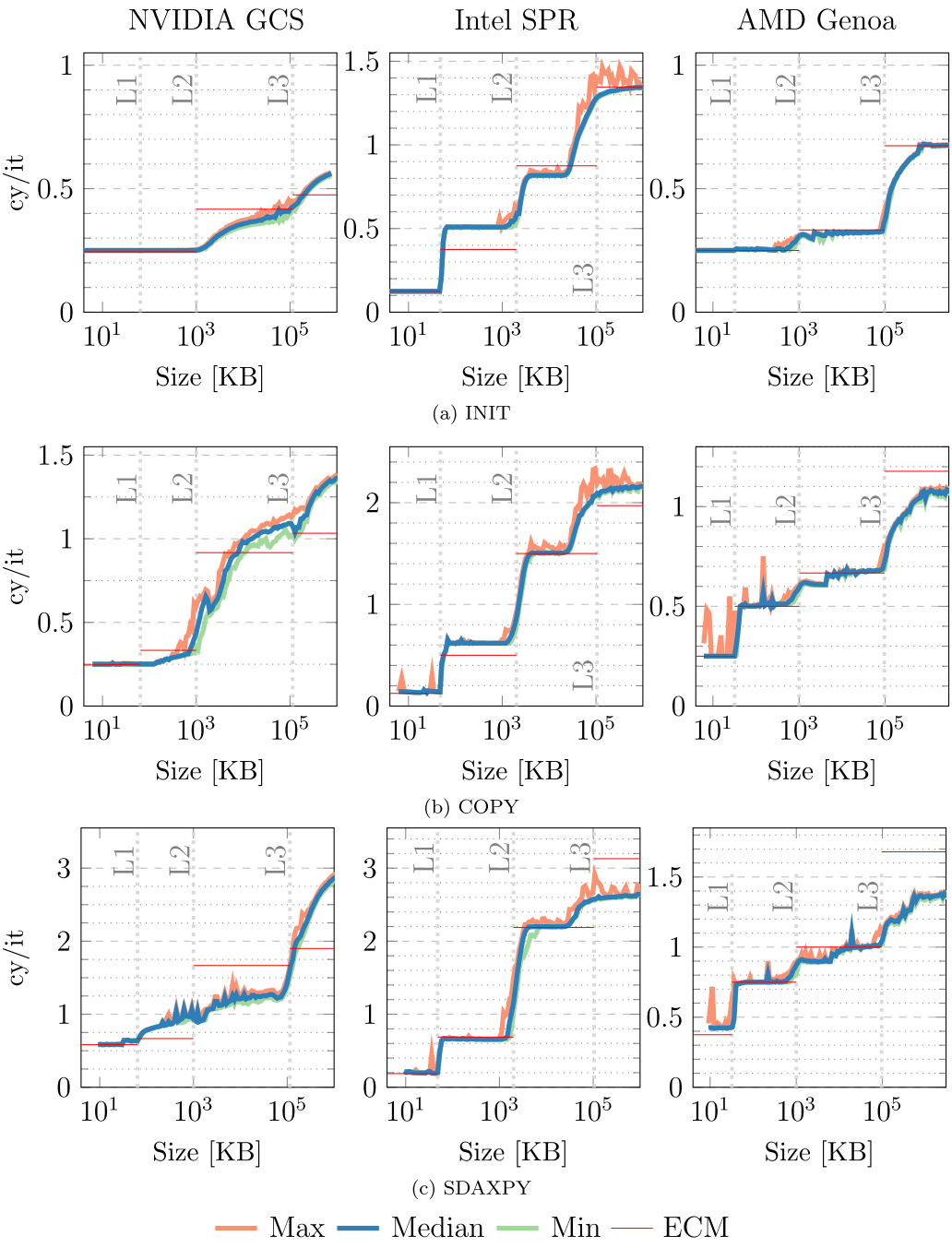


Fig. 7. Loop stream benchmarks for the (a) INIT, (b) COPY, and (c) SDAXPY kernels including the ECM prediction constructed in Section 3.3 in red. Each subgraph shows data for the GCS on the left, SPR in the middle and Genoa on the right. The time is normalized to cycles per double-precision element for the reader's convenience.

and Zen 4, respectively, established an in-core performance model for each of them, and applied it to simple streaming kernels. We showed that the models, incorporated in the Open Source Architecture Code Analyzer (OSACA), yield more accurate lower bounds for in-core runtime than the existing LLVM Machine Code Analyzer model for a comprehensive set of microbenchmarks. We also laid the foundation for an Execution-Cache-Memory (ECM) performance model for all three server models by investigating the memory hierarchy via streaming benchmark loops, indicating that the AMD Genoa cache hierarchies show largely overlapping characteristics in contrast to the Intel Sapphire Rapids, which aligns with earlier Intel architectures. NVIDIA Grace places itself somewhere in between with a partially overlapping cache hierarchy. Furthermore, we investigated the node-level capabilities of the HPC servers such as the sustained CPU clock frequencies and the memory bandwidth and focused on implicit and explicit write-allocate evasion techniques, concentrating on Intel Sapphire Rapids and its SpecI2M automatic cache line claim feature. We found that SpecI2M is more effective with a higher number of concurrent read streams in addition to the write stream, and that disabling Sub-NUMA Clustering on the CPU leads to better write-allocate evasion.

In future work, we plan to continue these investigations by improving our in-core model and extending the cache bandwidth measurements on all platforms to multi-core node-wide performance models, with the ultimate goal of applying the constructed ECM models to real applications.

CRediT authorship contribution statement

Jan Laukemann: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Georg Hager:** Writing – review & editing, Validation, Supervision, Conceptualization. **Gerhard Wellein:** Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was partly funded by BMBF, Germany through the DAREXA-F project.

References

[1] S. Williams, A. Waterman, D. Patterson, Roofline: An insightful visual performance model for multicore architectures, Commun. ACM 52 (4) (2009) 65–76, http://dx.doi.org/10.1145/1498765.1498785.

[2] F. Banchelli, J. Vinyals-Ylla-Catala, J. Pocurull, M. Clascà, K. Peiro, F. Spiga, M. Garcia-Gasulla, F. Mantovani, NVIDIA grace superchip early evaluation for HPC applications, in: Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region Workshops, in: HPCAsia '24 Workshops, Association for Computing Machinery, New York, NY, USA, 2024, pp. 45–54, http://dx.doi.org/10.1145/3636480.3637284.

[3] N.A. Simakov, M.D. Jones, T.R. Furlani, E. Siegmann, R.J. Harrison, First impressions of the NVIDIA grace CPU superchip and NVIDIA grace hopper superchip for scientific workloads, in: Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region Workshops, in: HPCAsia '24 Workshops, Association for Computing Machinery, New York, NY, USA, 2024, pp. 36–44, http://dx.doi.org/10.1145/3636480.3637097.

[4] J. Laukemann, J. Hammer, J. Hofmann, G. Hager, G. Wellein, Automated instruction stream throughput prediction for Intel and AMD microarchitectures, in: 2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems, PMBS, 2018, pp. 121–131, http://dx.doi.org/10.1109/PMBS.2018.8641578.

[5] J. Laukemann, J. Hammer, G. Hager, G. Wellein, Automatic throughput and critical path analysis of x86 and ARM assembly kernels, in: 2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems, PMBS, 2019, pp. 1–6, http://dx.doi.org/10.1109/PMBS49563.2019.00006.

[6] A. Abel, J. Reineke, Uica: Accurate throughput prediction of basic blocks on recent intel microarchitectures, in: L. Rauchwerger, K. Cameron, D.S. Nikolopoulos, D. Pnevmatikatos (Eds.), ICS '22: 2022 International Conference on Supercomputing, Virtual Event, USA, June 27-30, 2022, ICS '22, ACM, 2022, pp. 1–12, http://dx.doi.org/10.1145/3524059.3532396.

[7] A. Abel, S. Sharma, J. Reineke, Facile: Fast, accurate, and interpretable basic-block throughput prediction, in: 2023 IEEE International Symposium on Workload Characterization, IISWC, 2023, pp. 87–99, http://dx.doi.org/10.1109/IISWC59245.2023.00023.

[8] LLVM Compiler Infrastructure, LLVM machine code analyzer, 2024, [Online]. Available: https://llvm.org/docs/CommandGuide/llvm-mca.html. (Accessed 01 September 2024).

[9] A.S. Charif-Rubial, E. Oseret, J. Noudohouenou, W. Jalby, G. Lartigue, CQA: A code quality analyzer tool at binary level, in: 2014 21st International Conference on High Performance Computing, HiPC, 2014, pp. 1–10, http://dx.doi.org/10.1109/HiPC.2014.7116904.

[10] C. Mendis, A. Renda, S. Amarasinghe, M. Carbin, Ithemal: Accurate, portable and fast basic block throughput estimation using deep neural networks, in: International Conference on Machine Learning, PMLR, 2019, pp. 4505–4515, [Online]. Available: https://proceedings.mlr.press/v97/mendis19a.html.

[11] O. Sykora, P. Phothilimthana, C. Mendis, A. Yazdanbakhsh, GRANITE: A graph neural network model for basic block throughput estimation, in: 2022 IEEE International Symposium on Workload Characterization, IISWC, IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 14–26, http://dx.doi.org/10.1109/IISWC55918.2022.00012.

[12] J. Laukemann, T. Gruber, G. Hager, D. Oryspayev, G. Wellein, CloverLeaf on intel multi-core CPUs: A case study in write-allocate evasion, in: 2024 IEEE International Parallel and Distributed Processing Symposium, IPDPS, 2024, pp. 350–360, http://dx.doi.org/10.1109/IPDPS57955.2024.00038.

[13] J. Laukemann, G. Hager, G. Wellein, Microarchitectural comparison and in-core modeling of state-of-the-art CPUs: Grace, Sapphire Rapids, and Genoa, in: SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis, 2024, pp. 1405–1412, http://dx.doi.org/10.1109/SCW63240.2024.00181.

[14] Arm Limited, Arm Neoverse V2 Core Software Optimization Guide, Tech. Rep., Arm Limited, 2022, [Online]. Available: https://developer.arm.com/documentation/109898/latest/. (Accessed 01 September 2024).

[15] A.B. Yoo, M.A. Jette, M. Grondona, SLURM: Simple linux utility for resource management, in: D. Feitelson, L. Rudolph, U. Schwiegelshohn (Eds.), Job Scheduling Strategies for Parallel Processing, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 44–60, http://dx.doi.org/10.1007/10968987_3.

[16] T. Roehl, J. Treibig, G. Hager, G. Wellein, Overhead analysis of performance counter measurements, in: 43rd International Conference on Parallel Processing Workshops, ICCPW, 2014, pp. 176–185, http://dx.doi.org/10.1109/ICPPW.2014.34.

[17] T. Gruber, J. Eitzinger, G. Hager, G. Wellein, LIKWID, 2023, http://dx.doi.org/10.5281/zenodo.10105559.

[18] Intel® 64 and IA-32 architecture optimization reference manual, 2023, Intel Corporation, [Online]. Available: https://software.intel.com/en-us/download/intel-64-and-ia-32-architectures-optimization-reference-manual.

[19] Software optimization guide for the AMD Zen4 microarchitecture, 2023, AMD, [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/software-optimization-guides/57647.zip, Version 1.00.

[20] A. Abel, J. Reineke, Uops.info: Characterizing latency, throughput, and port usage of instructions on Intel microarchitectures, in: ASPLOS, ASPLOS '19, ACM, New York, NY, USA, 2019, pp. 673–686, http://dx.doi.org/10.1145/3297858.3304062.

[21] J. Hammer, G. Hager, G. Wellein, Ooo instruction benchmarking framework on the back of dragons, 2018, [Online]. Available: https://sc18.supercomputing.org/proceedings/src_poster/src_poster_pages/spost115.html, SC18 ACM SRC Poster.

[22] J. Hofmann, Ibench - instruction benchmarks, 2017, [Online]. Available: https://github.com/RRZE-HPC/ibench.

[23] J. Hofmann, C.L. Alappat, G. Hager, D. Fey, G. Wellein, Bridging the architecture gap: Abstracting performance-relevant properties of modern server processors, Supercomput. Front. Innov. 7 (2) (2020) 54–78, http://dx.doi.org/10.14529/jsfi200204.

[24] H. Stengel, J. Treibig, G. Hager, G. Wellein, Quantifying performance bottlenecks of stencil computations using the execution-cache-memory model, in: Proceedings of the 29th ACM on International Conference on Supercomputing, ICS '15, Association for Computing Machinery, New York, NY, USA, 2015, pp. 207–216, http://dx.doi.org/10.1145/2751205.2751240.

[25] J. Laukemann, G. Hager, Core-level performance engineering with the open-source architecture code analyzer (OSACA) and the compiler explorer, in: Companion of the 2023 ACM/SPEC International Conference on Performance Engineering, in: ICPE '23 Companion, Association for Computing Machinery, New York, NY, USA, 2023, pp. 127–131, http://dx.doi.org/10.1145/3578245.3583716.

[26] J.D. McCalpin, Memory bandwidth and machine balance in current high performance computers, in: IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, 1995, pp. 19–25, [Online]. Available: https://www.researchgate.net/publication/213876927_Memory_Bandwidth_and_Machine_Balance_in_Current_High_Performance_Computers.

[27] J. Hammer, Design and Implementation of an Automated Performance Modeling Toolkit for Regular Loop Kernels (Ph.D. thesis), Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU), 2023, http://dx.doi.org/10.25593/opus4-fau-21514.

[28] I.E. Papazian, New 3rd gen Intel® Xeon® scalable processor (codename: Ice lake-SP), in: Hot Chips Symposium, 2020, pp. 1–22, http://dx.doi.org/10.1109/HCS49909.2020.9220434.

[29] G. Hager, [Online]. Available: https://blogs.fau.de/hager/archives/8997.

[30] C. Alappat, N. Meyer, J. Laukemann, T. Gruber, G. Hager, G. Wellein, T. Wettig, Execution-cache-memory modeling and performance tuning of sparse matrix-vector multiplication and lattice quantum chromodynamics on A64FX, Concurr. Comput.: Pr. Exp. 34 (20) (2022) e6512, http://dx.doi.org/10.1002/cpe.6512.

[31] J. Hofmann, G. Hager, D. Fey, On the accuracy and usefulness of analytic energy models for contemporary multicore processors, in: R. Yokota, M. Weiland, D. Keyes, C. Trinitis (Eds.), High Performance Computing, Springer International Publishing, Cham, 2018, pp. 22–43, http://dx.doi.org/10.1007/978-3-319-92040-5_2.

[32] A. Limited, Arm Neoverse V2 Core Technical Reference Manual, Tech. Rep., Arm Limited, 2022, [Online]. Available: https://developer.arm.com/documentation/102375/latest/. (Accessed 10 July 2025).